Driving to a Point, a Line and a Pose

Contents

1. Overview 2
2. Driving to a point 4
   - 2.1 Aim at the dock and slow down near it 4
   - 2.2 The wheel limit: slowing both commands together 5
   - 2.3 Why the robot gets there: two errors that shrink 6
   - 2.4 What driving to a point cannot do: the arrival heading 8
3. Driving along a line 8
   - 3.1 Two errors from the line: the distance and the heading error 8
   - 3.2 Why it settles without swinging: a spring and a damper 10
   - 3.3 What line following cannot do: stop 11
4. Driving to a pose: the polar-coordinate controller 11
   - 4.1 Three numbers for a pose goal: a distance and two angles 12
   - 4.2 The rule: aim off to the side 13
   - 4.3 Why it works: the two angles near the dock 14
   - 4.4 Choosing the gains, and what breaks them 16
   - 4.5 Many starts, and a dock behind the robot 17
5. Where it runs: Nav2's graceful controller and docking 18
6. Summary 21
7. Sources 21
8. Key terms 22

Where this fits:

- **Builds on:** Differential-drive kinematics; Proportional control; Second-order systems: natural frequency and damping ratio.
- **Leads to:** Lateral and heading error.

1. Overview

Key point: To dock, the robot must stop at a place and face a direction: aiming at the place and slowing down near it arrives facing 64.0° instead of 90°, and following the dock's line faces the right way but never stops. The polar-coordinate controller steers on the distance and two angles at once and arrives at the dock facing 90.0°.

The robot can already hold one number at a time. Its speed loop holds 0.5 m/s on a ramp, its heading loop turns it to a new heading without swinging, and digital control runs those loops as code. None of them takes the robot to a place.

The robot of RO-001, a **differential drive** (G-2288) whose wheels each roll at most 0.6 m/s, stands at (2.0, 1.0) facing 30°. Its battery is low. The charging dock sits on the wall, and to charge, the robot must stop with its middle at (3.6, 3.5) and face the dock, 90°: that **pose** (G-2280), position plus heading, is the dock pose. Three numbers say how far the robot is from it:

- the dock point is 2.97 m away;
- it lies 27.4° to the left of where the robot faces;
- the robot must still turn 60° to face the dock.

Figure 1 tries the obvious rule: aim at the dock point and drive, slowing down near it. Watch the heading in the lower panel: the robot reaches the dock point, within 1 cm, after 8.10 s, but facing 64.0°, 26.0° short of 90°, so its contacts meet the dock at a slant. The last frame adds the controller this Note builds, which arrives facing 90.0°.

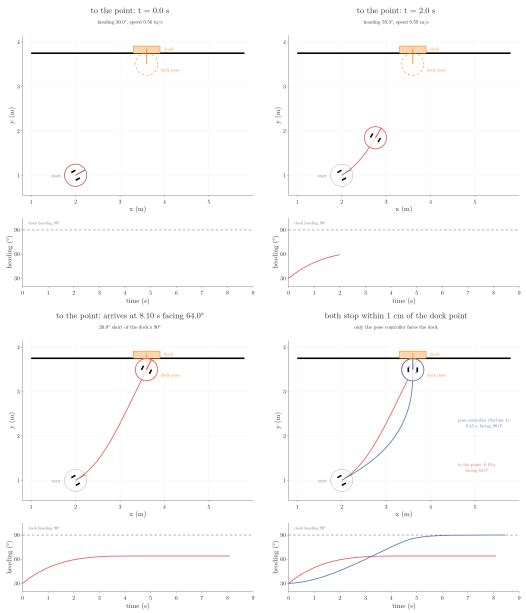


Figure 1: The robot (red) starts at (2.0, 1.0) facing 30° and aims at the dock point (3.6, 3.5), slowing down near it; one frame per 0.4 s. Top: the floor seen from above, with the wall, the dock and the dock pose (orange dashed, arrow at 90°). Bottom: the robot's heading over time against the dock's 90° (grey dashed). The robot stops within 1 cm of the dock point at 8.10 s facing 64.0°, 26.0° short. The last frame adds the polar-coordinate controller of Section 4 (blue): within 1 cm at 8.45 s, facing 90.0°

This Note builds the controller in three steps, each fixing what the last one could not do:

- driving to a point, and why it cannot choose the arrival heading (Section 2);
- driving along a line, which gets the heading right but cannot stop (Section 3);
- driving to a pose with the polar-coordinate controller, which does both (Section 4);
- where a pose controller runs in a real stack: Nav2's graceful controller, which its docking server uses (Section 5).

All three controllers command the robot's **linear velocity** (G-2291) v, its forward speed, and its **angular velocity** (G-2292) ω, its turn rate. A robot steered by these two numbers moves by the **kinematic model** (G-2297) of RO-001; any robot that drives forward and turns moves this way, and the model is then called the unicycle model. The robot's own fast loops deliver the commanded v and ω, the inner loops of a **cascade** (G-3086); the controllers here are the outer loop. They run at 20 Hz, the rate Nav2's controller server uses in its standard parameters, and the notebook `RO-069-driving-to-a-point-a-line-and-a-pose.ipynb` moves the robot in steps of 0.01 s.

2. Driving to a point

> **Key point:** The first part of the docking problem is to get to the dock point, 2.97 m away: driving forward in proportion to the distance and turning in proportion to the angle to the point gets there, because both shrink like first-order systems. But nothing in the rule mentions the dock's heading, so the robot arrives facing 64.0°.

2.1 Aim at the dock and slow down near it

To reach the dock point the robot needs a rule. How do we walk to a chair across a room? We turn towards it, walk, and slow down as we get close. The same two habits make a controller. The robot needs two numbers, both measured from where it stands to the dock point (Figure 4 in Section 4.1 draws them):

- ρ (rho) is the distance from the robot's middle to the dock point, in m, 2.97 m at the start;
- the **line of sight** (G-3121) is the straight line from the robot to the dock point, here at 57.4° from the x-axis; its direction comes from the **atan2** (G-2325) of the dock point's offset, rise 2.5 m and run 1.6 m;
- α (alpha) is the angle from the robot's **heading** (G-2284), its facing direction, to the line of sight, in rad, positive when the dock point is on the left: 57.4° − 30°, so 27.4° (0.4779 rad) at the start. Like every difference of angles, it is **wrapped** (G-2404) into −180° to 180°.

A distance and an angle that place a point are the point's **polar coordinates** (G-1511), here measured from the robot. The rule turns each into one command, by **proportional control** (G-2958), a command in proportion to an error (Corke RVC3 §4.1.1.1). The gains:

- k_ρ is the speed per metre of distance, 1 m/s per m (1 per s);
- k_α is the turn rate per radian of angle, 5 rad/s per rad (5 per s).

$$v = k_\rho \, \rho$$

$$\omega = k_\alpha \, \alpha$$

Steering by ρ and α like this is **driving to a point** (G-3120) (Corke RVC3 §4.1.1.1). At the start:

$$v = 1 \times 2.968 = 2.968 \text{ m/s}$$

$$\omega = 5 \times 0.4779 = 2.389 \text{ rad/s}$$

The robot turns left, towards the dock point, and the forward command is proportional to the distance, so the robot slows down as it gets close. But 2.968 m/s is five times what the wheels can do.

2.2 The wheel limit: slowing both commands together

The first command asks for more than the wheels can give. By the robot's **inverse kinematics** (G-2295), each wheel runs at v plus or minus ω times half the wheel separation, 0.1 m:

$$v_R = 2.968 + 2.389 \times 0.1 = 3.207 \text{ m/s}$$

$$v_L = 2.968 - 2.389 \times 0.1 = 2.729 \text{ m/s}$$

Cutting only the speed would change the arc the robot drives. Instead we shrink both commands by the same factor, the one that brings the faster wheel down to 0.6 m/s. The robot then drives the same arc, only more slowly, because the **turning radius** (G-2293) v divided by ω does not change. The symbols:

- c is the scale factor, a number with no unit;
- $v_{\max}$ is the largest wheel speed, 0.6 m/s;
- v_R is the faster wheel's commanded speed, 3.207 m/s.

$$c = 0.6/3.207 = 0.1871$$

$$v = 0.1871 \times 2.968 = 0.555 \text{ m/s}$$

$$\omega = 0.1871 \times 2.389 = 0.447 \text{ rad/s}$$

In symbols, scaling only when a wheel would pass the limit:

$$c = v_{\max}/v_R$$

$$v \leftarrow c\,v$$

$$\omega \leftarrow c\,\omega$$

The turning radius stays at 1.242 m before and after. The check at easy values: if the faster wheel already runs at exactly 0.6 m/s, the factor is 1 and nothing changes. Nav2's graceful controller keeps the arc in the same way when it caps the turn rate (Section 5). With the limit in place, the rule drives the robot to the dock point; Section 2.3 shows why it must get there.

2.3 Why the robot gets there: two errors that shrink

Does the rule really reach the dock point? Figure 2 plays the drive with ρ and α redrawn every 0.25 s. Watch the red arc: α falls from 27.4° to 11.6° in the first second and is gone by about 4 s, while ρ shrinks steadily. From then on the robot drives straight at the dock point.

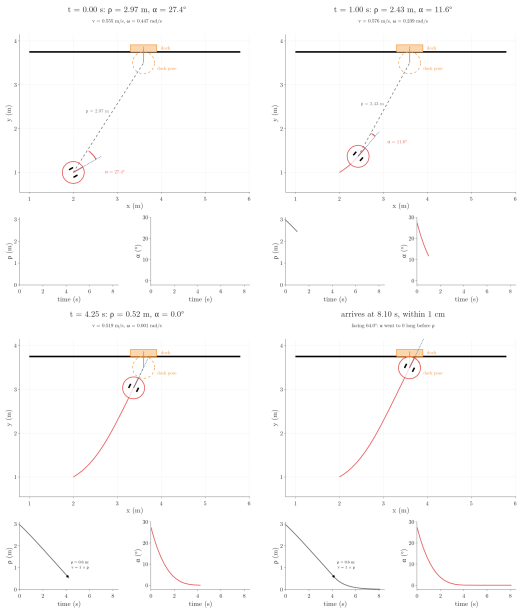


Figure 2: Driving to the dock point with forward speed v = 1 × ρ and turn rate $\omega = 5$ α, scaled together at the wheel limit; one frame per 0.25 s. The grey dashed line is ρ, the distance to the dock point; the red arc is α, from the robot's heading (blue dotted) to the line of sight. Lower panels: ρ and α over time. α is near 0 by 4 s; ρ falls at about 0.6 m/s until it reaches 0.6 m at 4.11 s (black dot), then shrinks to about a third every second. The last frame: within 1 cm at 8.10 s, facing 64.0°

Why do both shrink? Two rates decide it, and each follows from how the robot's velocity, v along its heading, splits against the line of sight, the same split into a cosine and a sine part that gives the robot's motion along x and y:

- the part along the line of sight, v times the cosine of α, closes the distance, so ρ falls at that rate;
- the part across it, v times the sine of α, swings the line of sight round the dock point. At distance ρ, a sideways speed turns the line at that speed divided by ρ, by RO-001's arc rule, speed equals turn rate times distance;
- α is the line of sight's direction minus the heading, so it changes by the line's swing minus the turn rate ω.

The symbols, with a dot for a rate of change, the **dot notation** (G-2296):

- $\dot{\rho}$ is the rate of change of the distance, in m/s;
- $\dot{\alpha}$ is the rate of change of α, in rad/s.

$$\dot{\rho} = -v \cos \alpha$$

$$\dot{\alpha} = \frac{v \sin \alpha}{\rho} - \omega$$

Near the dock point the speed is k_ρ times ρ (no wheel limit below 0.6 m) and α is small, so its cosine is about 1 and its sine about α itself. Our numbers first, at $\rho = 0.5$ m and $\alpha = 0.1$ rad, where $v = 0.5$ m/s and $\omega = 0.5$ rad/s, then the same in symbols:

$$\dot{\rho} = -0.5 \times \cos 0.1 = -0.4975$$

$$\dot{\rho} \approx -1 \times 0.5$$

$$\dot{\alpha} = 0.5 \times \sin 0.1/0.5 - 0.5$$

$$= 0.0998 - 0.5 = -0.4002$$

$$\dot{\alpha} \approx (1 - 5) \times 0.1$$

$$\dot{\rho} \approx -k_\rho \, \rho$$

$$\dot{\alpha} \approx (k_\rho - k_\alpha) \, \alpha$$

Each error shrinks at a rate in proportion to its own size, which makes each a **first-order system** (G-2987), the kind that closes the same fraction of its gap in every **time constant** (G-2988). Choosing the commands so that each error obeys such an equation, and so decays to zero, is the standard way to build the rule (COS 495 L6, slide 9). Here:

- ρ has time constant 1 divided by k_ρ, 1 s. Figure 2 shows it: ρ reaches 0.6 m at 4.11 s and 0.215 m one second later, 36 % of 0.6 m. A smooth loop would keep 37 %; holding each command for 0.05 s, the **zero-order hold** (G-3109) of a 20 Hz loop, costs the difference.
- α shrinks only if k_α is larger than k_ρ: otherwise the line of sight swings away faster than the robot turns after it. With 5 and 1, α has time constant 1/4 s, so it dies out long before ρ.

So the robot reaches the dock point, as Figure 2 shows: within 1 cm after 8.10 s. Its heading is a different matter.

2.4 What driving to a point cannot do: the arrival heading

Once α is gone, by 4 s, the robot drives straight along the line of sight, and the heading it has then is the heading it arrives with: 64.0°, the last frame of Figure 2. The rule has no number for the dock's heading, so nothing pulls the heading towards 90°. The arrival heading depends only on where the robot started and how fast it turned: with $k_\alpha = 3$ instead of 5, the same start arrives facing 70.7° (notebook).

One fix is to stop at the dock point and spin 26.0° on the spot: position first, heading second. The robot has two inputs for the three numbers of its pose, so a two-stage plan is a natural one, and Nav2's graceful controller uses it by default at the end of a path (`prefer_final_rotation`; its README: the robot follows the path's direction and makes "a final rotation in place to the goal orientation"). Our robot can spin on the spot, but it must stop to do so, and a car cannot spin at all. A better way gets the heading right on the way in. The dock's heading defines a line, and Section 3 makes the robot follow it.

3. Driving along a line

> **Key point:** The second part of the docking problem is the arrival heading: steering on the distance from the dock's line and on the heading error, how far the heading points away from the line, brings the robot onto the line facing 90° without swinging, like a block on a spring and a damper. But nothing in the rule says where to stop, so the robot meets the dock at 0.50 m/s.

3.1 Two errors from the line: the distance and the heading error

A robot that arrives along the dock's line arrives facing the dock. The dock's line runs through the dock point at 90°: the line x = 3.6 m, followed upwards. The robot needs two numbers about it, both shown in Figure 3:

- e is the signed distance from the robot's middle to the line, in m, positive when the robot is to the line's left: 1.60 m at the start;
- ψ (psi) is the heading error, the robot's heading minus the line's direction, in rad, wrapped: 30° − 90°, so −60° (−1.0472 rad) at the start.

A driver joining a lane does the same: steer towards the lane in proportion to how far off it the car is, and straighten up in proportion to how far the car points away from the lane's direction. The robot keeps a fixed forward speed and steers on both errors (Corke RVC3 §4.1.1.2):

- v is the forward speed, 0.5 m/s;
- K_d is the turn rate per metre of distance, 2 rad/s per m;
- K_h is the turn rate per radian of heading error, 2 per s.

$$v = 0.5 \text{ m/s}$$

$$\omega = -K_d\, e - K_h\, \psi$$

The minus signs make the robot turn back towards the line: right when it is left of the line, right when it points left of the line's direction. Steering on e and ψ with a fixed speed is **driving along a line** (G-3122) (Corke RVC3 §4.1.1.2). At the start:

$$\omega = -2 \times 1.60 - 2 \times (-1.0472)$$

$$= -3.2000 + 2.0944$$

$$= -1.1056 \text{ rad/s}$$

The distance term asks for a right turn of 3.200 rad/s and the heading term, since the robot already points 60° to the right of the line's direction, asks for 2.094 rad/s back to the left. The distance wins, so the robot turns right first, down to a heading of 13.7°, and approaches the line at a steeper angle. The faster wheel would need 0.61056 m/s, so the wheel limit of Section 2.2 scales both commands by 0.9827, to 0.491 m/s and −1.086 rad/s.

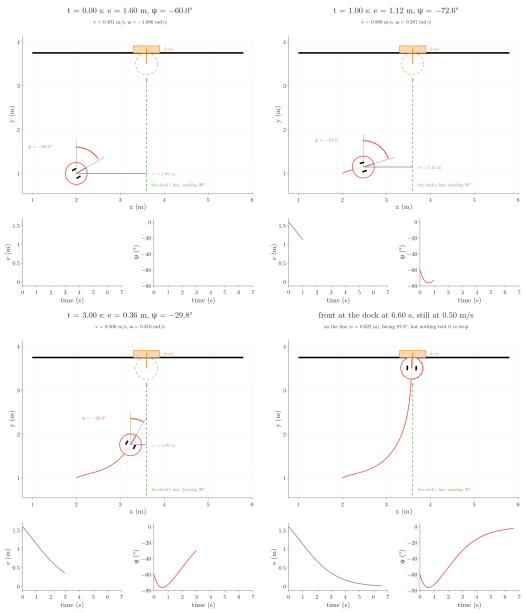


Figure 3: Driving along the dock's line x = 3.6 m (green dashed) at 0.5 m/s with turn rate $\omega = -2$ e $- 2\ \psi$; one frame per 0.25 s. The purple arrow is e, the distance from the line; the red arc is ψ, from the line's direction (green dotted) to the robot's heading (blue dotted). Lower panels: e and ψ over time. The robot first turns right to 13.7°, then curves onto the line without crossing it. The last frame: its front meets the dock at 6.60 s, still at 0.50 m/s, 0.022 m from the line and facing 87.9°

The robot joins the line without crossing it; Section 3.2 shows why it does not swing past.

3.2 Why it settles without swinging: a spring and a damper

Figure 3 shows the robot curving onto the line without crossing it. Why no swing? Near the line both errors are small. The distance changes with the part of the velocity square to the line, v times the sine of ψ, about v times ψ, and the heading error changes at the turn rate, because the line's direction is fixed. Then, with numbers first:

$$\dot{e} \approx 0.5\,\psi$$

$$\dot{\psi} = \omega = -2\,e - 2\,\psi$$

$$\ddot{e} \approx 0.5\,\dot{\psi}$$

$$= 0.5 \times (-2\,e - 2\,\psi)$$

$$= -1\,e - 2\,\dot{e}$$

$$\ddot{e} + 2\,\dot{e} + 1\,e = 0$$

The same steps in symbols, with $\ddot{e}$ the rate of change of $\dot{e}$:

$$\dot{e} = v \sin\psi \approx v\,\psi$$

$$\dot{\psi} = -K_d\,e - K_h\,\psi$$

$$\ddot{e} \approx v\,\dot{\psi}$$

$$= -vK_d\,e - K_h\,v\psi$$

$$\ddot{e} + K_h\,\dot{e} + vK_d\,e = 0$$

The last line is the **error dynamics** (G-2986) of a block on a spring and a damper, a **second-order system** (G-2990): the speed times K_d is the spring, 1, and K_h is the damper, 2. In RO-061's form, with the **natural frequency** (G-2991) ω_n and the **damping ratio** (G-2992) ζ:

$$\omega_n^2 = 0.5 \times 2 = 1$$

$$\omega_n = 1 \text{ rad/s}$$

$$2\zeta\omega_n = 2$$

$$\zeta = 2/(2 \times 1) = 1$$

A damping ratio of 1 is **critically damped** (G-2995): the fastest reply that does not overshoot, and the reason we chose $K_h = 2$ for $K_d = 2$ at 0.5 m/s. The spring hides a condition: it is the speed times K_d, so the same gains swing at a higher speed. At 2 m/s:

$$\omega_n^2 = 2 \times 2 = 4$$

$$\zeta = 2/(2 \times 2) = 0.5$$

The robot settles onto the line facing 90°: at 6 s it is 0.036 m from the line and facing 86.6°. Line following has fixed the heading, but it has broken something else.

3.3 What line following cannot do: stop

The rule drives at 0.5 m/s whatever happens, and the dock point is not in it. In the last frame of Figure 3 the robot's front meets the dock at 6.60 s at full speed, 0.022 m from the line and facing 87.9°. A line has no end, so a line follower has nothing to stop at. Driving along a line is the seed of the next Note's job, following a path that has no end in view; for a dock we need both: Section 2's slowing distance and Section 3's heading, in one rule.

4. Driving to a pose: the polar-coordinate controller

> **Key point:** The whole docking problem asks for the place and the heading together: adding one angle, β, from the line of sight to the dock's heading, and turning against it, makes the robot swing out and come in along the dock's line. Near the dock the two angles behave like a critically damped spring and damper, and the robot arrives at 8.45 s facing 90.0°.

4.1 Three numbers for a pose goal: a distance and two angles

Driving to a point used ρ and α; driving along a line used a heading error. To stop at the dock facing 90°, the robot needs one more number that says how far the dock's heading is from the way the robot is approaching. The new number is β (beta), drawn in Figure 4 at the start:

- ρ is the distance to the dock point, 2.97 m (Section 2.1);
- α is the angle from the robot's heading to the line of sight, the straight line to the dock point, 27.4°;
- β is the angle from the line of sight to the dock's heading, in rad, wrapped: 90° − 57.4°, so 32.6° (0.5693 rad) at the start.

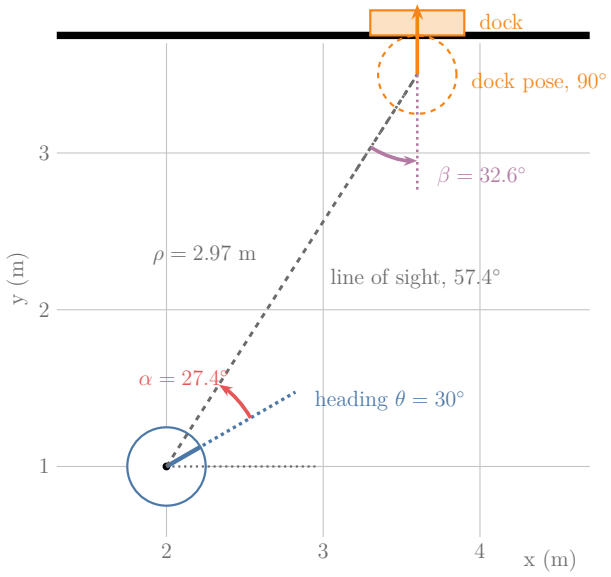


Figure 4: The three numbers of the polar-coordinate controller at the start. The robot (blue disk) at (2.0, 1.0) faces 30° (blue dotted); the grey dashed line of sight to the dock point runs at 57.4° and is $\rho = 2.97$ m long; the red arc $\alpha = 27.4°$ turns from the heading to the line of sight. At the dock point the purple arc $\beta = 32.6°$ turns from the line of sight to the dock's heading of 90°; both lines are drawn continued backwards through the dock point, below it, which gives the same angle

When the robot is at the dock facing 90°, all three are 0. The robot's heading is the line of sight's direction minus α, and the dock's heading is the line of sight's direction plus β. So the heading still to turn, 60° at the start, is α plus β:

$$27.4° + 32.6° = 60.0°$$

Driving all three to 0 brings the robot to the dock point facing 90°. Bringing a robot to a given position and heading is **driving to a pose** (G-3123) (Corke RVC3 §4.1.1.4). Section 4.2 builds a rule that drives all three to 0.

4.2 The rule: aim off to the side

How does β help? Picture the dock's line. The robot starts to its left, so to come in along the line it must first move right, across in front of the dock, and only then turn towards it. Moving right swings the line of sight from 57.4° towards 90°, which shrinks β. So the rule turns the robot to the right in proportion to β, on top of turning it towards the dock point in proportion to α (Corke RVC3 §4.1.1.4; PythonRobotics, Move to a Pose). The gains:

- k_ρ is the speed per metre of distance, 1 per s, as in Section 2;
- k_α is the turn rate per radian of α, 5 per s, as in Section 2;
- k_β is the turn rate per radian of β, -4 per s; it is negative, so a positive β asks for a right turn.

$$v = k_\rho \, \rho$$

$$\omega = k_\alpha \, \alpha + k_\beta \, \beta$$

The rule is the **polar-coordinate controller** (G-3124), named after the distance and angles it steers on (Siegwart and Nourbakhsh §3.6, through COS 495 L6). Only k_β is new: the only change from Section 2's rule is the β term. At the start:

$$5 \times 0.4779 = 2.389 \text{ rad/s}$$

$$-4 \times 0.5693 = -2.277 \text{ rad/s}$$

$$\omega = 2.389 - 2.277 = 0.112 \text{ rad/s}$$

$$v = 1 \times 2.968 = 2.968 \text{ m/s}$$

The two angle terms almost cancel: aim at the dock point, but not yet, because the dock's heading is far from the line of sight. The wheel limit of Section 2.2 scales both commands by 0.2014, to 0.598 m/s and 0.023 rad/s, so the robot first drives nearly straight on at 30°, to the right of the line of sight.

Figure 5 plays the drive with all three numbers redrawn every 0.25 s. Watch the purple arc at the dock: β shrinks as the robot moves right of the line of sight, and the red arc α grows at first, to 28.4° at 1 s, before both fall together.

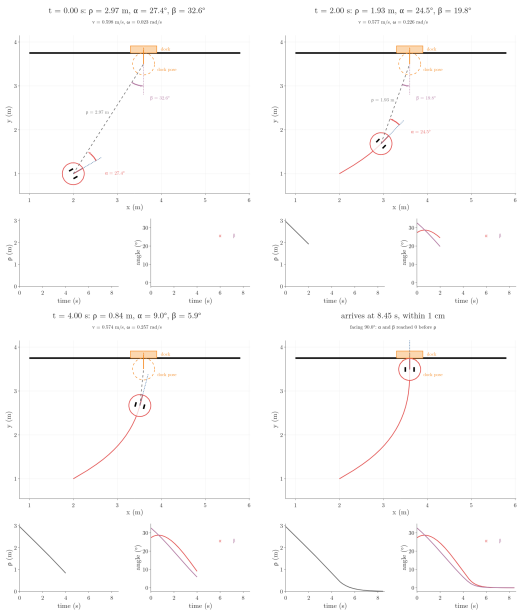


Figure 5: Driving to the dock pose with forward speed v = 1 × ρ and turn rate ω = 5 α − 4 β, scaled together at the wheel limit; one frame per 0.25 s. Grey dashed: ρ; red arc at the robot: α; purple arc at the dock point: β (the line of sight and the dock's heading both continued backwards through the dock point). The arcs are left out within 1.2 m of the dock. Lower panels: ρ, and α (red) and β (purple) over time. The robot keeps right of the line of sight, so β shrinks first; α and β then fall to 0 together. The last frame: within 1 cm at 8.45 s, facing 90.0°

The three numbers once a second (notebook):

t (s)	ρ (m)	α (°)	β (°)	heading (°)
0	2.968	27.38	32.62	30.00
1	2.448	28.40	26.64	34.95
2	1.930	24.47	19.77	45.75
4	0.836	8.97	5.86	75.17
6	0.118	0.37	0.21	89.41
8	0.015	0.01	0.01	89.99

The robot stops within 1 cm of the dock point at 8.45 s facing 90.0°, 0.35 s later than driving to the point, and with the heading right. Why does it work, and why these gains?

4.3 Why it works: the two angles near the dock

The table shows α and β reaching 0 together, before ρ does. To see why, take the rates of the three numbers. ρ and α change as in Section 2.3. β is the dock's fixed heading minus the line of sight's direction, so it changes by minus the line's swing. The symbols are those of Section 4.1 and Section 4.2, with $\dot{\beta}$ the rate of change of β, in rad/s (COS 495 L6, slides 22 to 25):

$$\dot{\rho} = -v \cos \alpha$$

$$\dot{\alpha} = \frac{v \sin \alpha}{\rho} - \omega$$

$$\dot{\beta} = -\frac{v \sin \alpha}{\rho}$$

Near the dock, v is k_ρ times ρ and both angles are small, so the sine of α is about α. Put in the rule, our gains first, with $\ddot{\beta}$ the rate of change of $\dot{\beta}$:

$$\dot{\alpha} \approx 1\,\alpha - 5\,\alpha + 4\,\beta$$

$$= -4\,\alpha + 4\,\beta$$

$$\dot{\beta} \approx -1\,\alpha$$

$$\ddot{\beta} \approx -1\,\dot{\alpha}$$

$$= 4\,\alpha - 4\,\beta$$

$$= -4\,\dot{\beta} - 4\,\beta$$

$$\ddot{\beta} + 4\,\dot{\beta} + 4\,\beta = 0$$

The same steps in symbols:

$$\dot{\alpha} \approx k_\rho \alpha - k_\alpha \alpha - k_\beta \beta$$

$$\dot{\beta} \approx -k_\rho\,\alpha$$

$$\ddot{\beta} \approx -k_\rho\,\dot{\alpha}$$

$$= (k_\alpha - k_\rho)\,k_\rho \alpha + k_\rho k_\beta\,\beta$$

$$= -(k_\alpha - k_\rho)\,\dot{\beta} + k_\rho k_\beta\,\beta$$

$$\ddot{\beta} + (k_\alpha - k_\rho)\,\dot{\beta} - k_\rho k_\beta\,\beta = 0$$

Again a block on a spring and a damper, as in Section 3.2, and α obeys the same equation. The spring is minus k_ρ times k_β, 4; the damper is k_α minus k_ρ, 4. In RO-061's form:

$$\omega_n^2 = -1 \times (-4) = 4$$

$$\omega_n = 2 \text{ rad/s}$$

$$\zeta = 4/(2 \times 2) = 1$$

A critically damped pair with a natural frequency of 2 rad/s is the reply of RO-061's chosen turn, which settles within 2 % in 2.92 s. Since α and β both go to 0, so does the heading still to turn, their sum, and the robot faces 90°.

A block stays near its rest position only if the spring pulls it back and the damper resists its motion: both must be positive (COS 495 L6, slide 28). Requiring both gives the three conditions on the gains, with the distance's own rate k_ρ positive so that ρ shrinks:

$$k_\rho > 0$$

$$k_\beta < 0$$

$$k_\alpha - k_\rho > 0$$

The second condition says the β term must turn the robot away from the dock's heading, as in Section 4.2; the third is Section 2.3's condition again: turning must outpace the swing of the line of sight. Section 4.4 picks gains that meet all three, and breaks each in turn.

4.4 Choosing the gains, and what breaks them

Why 1, 5 and −4? Three reasons, one per gain:

- $k_\rho = 1$ per s sets how fast the robot closes the last 0.6 m, with a time constant of 1 s (Section 2.3).
- The angles must settle faster than the distance, or the robot reaches the dock point still turning. A natural frequency of 2 rad/s, twice k_ρ, does it, and gives k_β: minus 4 divided by 1.

- A damping ratio of 1 gives the fastest settling without a swing past the dock's line, and gives k_α: the damper, 4, plus k_ρ.

The second reason shows when it is broken. With $k_\alpha = 3$ and $k_\beta = -1$, which meet all three conditions but give a natural frequency of only 1 rad/s, as fast as the distance, eight starts round the dock arrive facing anywhere from 84.7° to 95.3° (notebook).

Figure 6 breaks one condition at a time from the same start. Watch the red robot: with $k_\beta = +4$, the spring pushes instead of pulling, the robot turns towards the dock's heading too early and its front hits the wall at 4.70 s, 0.79 m left of the dock. The purple robot has $k_\alpha = 0.5$, less than k_ρ, so the damper pushes too: it circles and, after 30 s, is still 1.87 m from the dock.

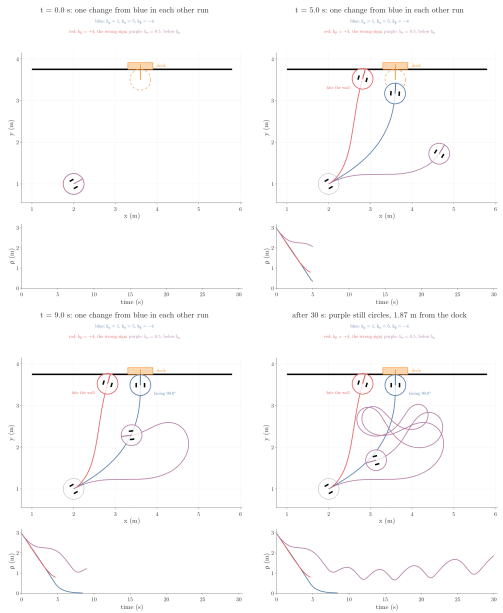


Figure 6: Three sets of gains from the same start, the gains kρ, kα and kβ on the distance ρ and the angles α and β; one frame per 0.5 s for 30 s. Blue: k$\rho = 1$, k$\alpha = 5$, k$\beta = -4$, all three conditions met: within 1 cm of the dock at 8.45 s, facing 90.0°. Red: k$\beta = +4$, the wrong sign: its front hits the wall at 4.70 s, 0.79 m left of the dock. Purple: k$\alpha = 0.5$, below kρ: it circles and is still 1.87 m from the dock after 30 s. Lower panel: the distance ρ of each over time

The chosen gains work from our one start; Section 4.5 tries eight others.

4.5 Many starts, and a dock behind the robot

A controller for docking must work from wherever the robot happens to be. Figure 7 starts eight robots on a half circle of radius 2 m below the dock, every 20° from 200° to 340°, each facing 90°, with the same gains. Each run is on its own; the eight are drawn together. Watch the robots at the ends: the dock point lies 70° to their side, so they first turn towards it, then swing back and come in along the dock's line.

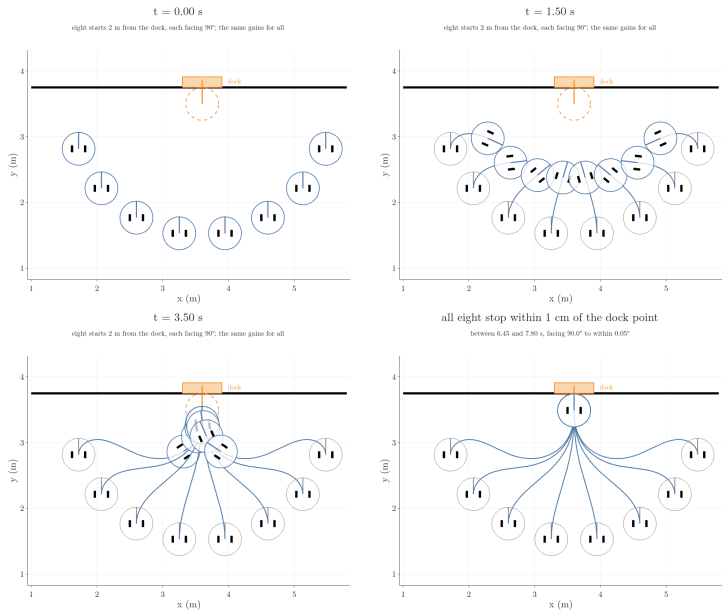


Figure 7: The polar-coordinate controller (gains k$\rho = 1$, k$\alpha = 5$, k$\beta = -4$ on the distance ρ and the angles α and β) from eight starts 2 m from the dock, every 20° from 200° to 340°, each facing 90°; one frame per 0.25 s. Grey: the starts; blue: the robots and their paths. Each run is separate. The last frame: all eight stop within 1 cm of the dock point between 6.45 and 7.80 s, facing 90.0° to within 0.05°

All eight stop within 1 cm of the dock point between 6.45 and 7.80 s, facing 90.0° to within 0.05°.

One case breaks the rule as written: a dock point behind the robot, with α beyond ±90°. Then the part of the velocity along the line of sight points away from the dock, and the linear analysis of Section 4.3, which assumed small angles, does not hold. The standard fix drives the robot backwards: measure α from the robot's back instead of its front and make v negative (COS 495 L6, slides 30 to 33). Corke's code picks the direction once, at the start, and PythonRobotics' code picks it at every step. A real navigation stack docks with a controller of the same family, which Section 5 runs on our dock.

5. Where it runs: Nav2's graceful controller and docking

> **Key point:** Nav2, the navigation software of ROS 2, docks robots with a pose controller of the same family: its graceful controller steers on the distance and two angles to a target pose, and sets the speed from how tightly the robot must turn. On our dock it arrives facing 90° to within 0.1°, more slowly than our controller.

The docking problem of the Overview is a standard task, and Nav2 solves it with a pose controller. Its package `nav2_graceful_controller` implements the smooth control law of Park and Kuipers (ICRA 2011), named in its README. A controller that drives to a pose by choosing the curvature of the arc to drive from the distance and two angles, and the

speed from that curvature, is a **graceful controller** (G-3125). In Nav2's code (the files `ego_polar_coords.hpp` and `smooth_control_law.cpp` of the package, read for this Note):

- It measures the same three numbers. Nav2 calls them r, the distance (our ρ); ϕ, the target's heading minus the line of sight, the line from the robot to the target (our β); and δ, the robot's heading minus the line of sight (our α with the opposite sign).
- Instead of a turn rate, it chooses the curvature of the arc to drive. The heading is steered towards an angle off the line of sight that grows with ϕ but is capped by an arctangent, so a large ϕ cannot ask for more than a quarter turn; this is the "slow subsystem" of the README. The heading's own error is fed back with gain `k_delta`, the "fast subsystem".
- The speed is the largest speed divided by 1 plus `beta` times the curvature squared (`lambda` = 2), so the robot slows on tight arcs. Within `slowdown_radius` of the target it slows further in proportion to the distance, and the turn rate is the curvature times the speed, capped at `v_angular_max` with the speed cut to keep the arc, as in Section 2.2.

The defaults in `src/parameter_handler.cpp` are `k_phi` 2.0, `k_delta` 1.0, `beta` 0.4, `lambda` 2.0, `v_linear_max` 0.5 m/s, `v_linear_min` 0.1 m/s, `v_angular_max` 1.0 rad/s and `slowdown_radius` 1.5 m. The package's README still lists a `motion_target_dist` that the code has replaced by `min_lookahead` 0.25 m and `max_lookahead` 1.0 m (`src/graceful_controller.cpp`): as a path follower, the controller takes as its target the pose on the planned path `max_lookahead` ahead, or a nearer one if that one cannot be reached without a collision, and drives to it again at every cycle. The docking server (`controller.cpp` of the package `opennav_docking`, in Nav2's `nav2_docking` folder) builds the same control law with gentler numbers for the last approach: `k_phi` 3.0, `k_delta` 2.0, `v_linear_max` 0.25 m/s and `slowdown_radius` 0.25 m.

Figure 8 runs Nav2's law, with the plugin's defaults, on our dock next to our controller. Watch the speed panel: the graceful controller starts below 0.5 m/s, slows from 3.60 s, 1.38 m from the dock, where its slow-down near the target (which starts 1.5 m out) first asks for less than the 0.47 m/s it was driving, and creeps in at its 0.1 m/s floor.

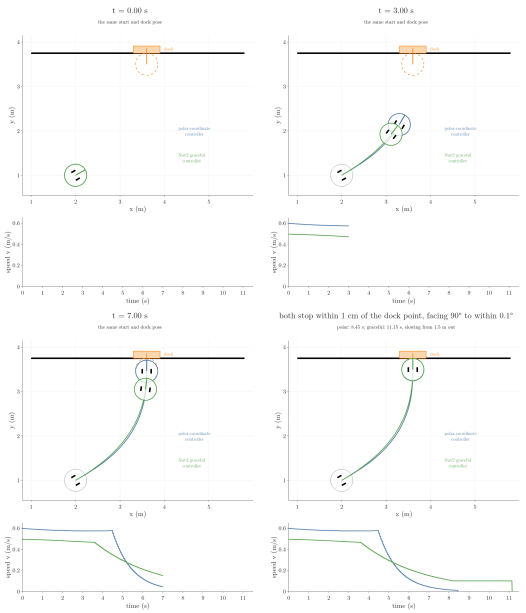


Figure 8: Nav2's graceful controller (green, its SmoothControlLaw with the plugin's defaults listed above: at most 0.5 m/s and 1.0 rad/s, slowing within 1.5 m) and this Note's polar-coordinate controller (blue) from the same start to the same dock pose; one frame per 0.25 s. Bottom: the forward speed of each. The paths almost coincide; the graceful controller drives at most 0.495 m/s, slows near the dock (its slow-down starts 1.5 m out and takes over at 1.38 m, at 3.60 s) and finishes at its 0.1 m/s floor. The last frame: both within 1 cm of the dock point, facing 90° to within 0.1°; polar at 8.45 s, graceful at 11.15 s

Both arrive facing 90°, the graceful controller at 11.15 s against 8.45 s, along almost the same path. The difference is the speed rule: the graceful controller slows on tight arcs and well before the target, which is what "graceful" means in the paper's title: in the README's words, "a smooth and comfortable trajectory". Open versions of the polar-coordinate controller itself are Corke's `driveconfig.py` and PythonRobotics' `move_to_pose.py`.

6. Summary

Controller	Steers on	Speed	At the dock
Driving to a point	ρ, α	k_ρ times ρ	8.10 s, facing 64.0°
Driving along a line	e, ψ	fixed, 0.5 m/s	meets it at 6.60 s at 0.50 m/s
Polar-coordinate (pose)	ρ, α, β	k_ρ times ρ	8.45 s, facing 90.0°
Nav2 graceful	r, ϕ, δ	from the curvature	11.15 s, facing 90° to within 0.1°

- The opening question has its answer: from (2.0, 1.0) facing 30°, the polar-coordinate controller stops within 1 cm of the dock point (3.6, 3.5) at 8.45 s facing 90.0°, because it steers on the distance ρ and two angles, α to the line of sight and β from it to the dock's heading.
- **Driving to a point** (G-3120) sets the speed in proportion to the distance ρ and the turn rate in proportion to the angle α, so both shrink like first-order systems, with time constants 1 s and 1/4 s; but nothing in it names the dock's heading, so the robot arrives facing 64.0°.
- A command beyond the wheel limit is scaled down, speed and turn rate by the same factor, so the robot drives the same arc more slowly instead of a different one.
- **Driving along a line** (G-3122) steers on the distance e from the line and the heading error ψ, so the robot settles onto the dock's line like a critically damped spring and damper; but a line has no end, so it meets the dock at 0.50 m/s.
- The **polar-coordinate controller** (G-3124) adds the term in β, so the robot swings out and comes in along the dock's line; near the dock the angles obey a spring-and-damper equation, which settles only when k_ρ is positive, k_β negative and k_α larger than k_ρ.
- Choosing $k_\beta = -4$ and $k_\alpha = 5$ makes the angles critically damped at 2 rad/s, twice the distance's rate, so the heading is right before the robot arrives: eight starts all face 90.0° to within 0.05°.
- Nav2's **graceful controller** (G-3125) drives to a pose from the same three numbers and slows on tight arcs and near the target, so its docking approach is smooth; on our dock it arrives at 11.15 s.

The robot can now drive to any pose on an open floor. Still missing: a planned route is not one pose but a long curve, and a car cannot spin to fix its heading; following a path means measuring, at every moment, how far the car is from the path and how its heading differs from the path's, which is path following and its errors.

7. Sources

Built from

- A. Sakai et al., PythonRobotics, "Move to a Pose Control", documentation page and code `move_to_pose.py`, https://atsushisakai.github.io/PythonRobotics/modules/6_path_tracking/move_

(PythonRobotics)

- NPTEL-NOC IITM, *Wheeled Mobile Robots*, lecture 37, "Simulation of Land-based Mobile Robots along with Kinematic Control Part 2", YouTube, https://www.youtube.com/watch?v=1uH (NPTEL L37)
- Corke, P. (2023). *Robotics, Vision and Control: Fundamental Algorithms in Python*, 3rd ed., Springer. §4.1.1.1 "Driving to a Point", §4.1.1.2 "Driving Along a Line", §4.1.1.4 "Driving to a Configuration"; the book's open code: notebook `chap4.ipynb` and models `drivepoint.py`, `driveline.py`, `driveconfig.py`, https://github.com/petercorke/RVC3-python (Corke RVC3)
- Clark, C., COS 495 *Autonomous Robot Navigation*, Lecture 6 "Point Tracking", Princeton University, 2011, figures after Siegwart and Nourbakhsh, https://www.cs.princeton.edu/courses/archive/fall11/cos495/COS495-Lecture6-PointTracking.pdf (COS 495 L6)
- Siegwart, R. and Nourbakhsh, I. R. (2004). *Introduction to Autonomous Mobile Robots*, MIT Press, §3.6 "Motion control (kinematic control)", through the slides above (Siegwart and Nourbakhsh §3.6)

Other references

- Park, J. J. and Kuipers, B. (2011). "A smooth control law for graceful motion of differential wheeled mobile robots in 2D environment", ICRA 2011, as named in Nav2's README
- Open Navigation, Nav2, package `nav2_graceful_controller`: its README and the files `ego_polar_coords.hpp`, `smooth_control_law.cpp`, `parameter_handler.cpp` and `graceful_controller.cpp`; folder `nav2_docking`: the file `src/controller.cpp` of the package `opennav_docking`, https://github.com/ros-navigation/navigation2 (read for the law and the defaults quoted in Section 5)

8. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Line of sight (to a goal) (G-3121)	The straight line from a robot to its goal point; its direction minus the robot's heading is the angle the robot must turn to face the goal, the angle a point or pose controller steers on.

Term	Meaning
Driving to a point (G-3120)	Steering a robot to a goal point by setting its forward speed in proportion to the distance to the point and its turn rate in proportion to the angle between its heading and the line of sight, $v = k_\rho \rho$, $\omega = k_\alpha \alpha$; the robot slows down as it nears the point and gets there, but arrives with whatever heading it happens to have.
Driving along a line (G-3122)	Steering a robot onto a straight line at a fixed speed, with a turn rate set by its signed distance from the line and its heading error against the line's direction, $\omega = -K_d e - K_h \psi$; the robot settles onto the line like a spring and damper, but a line has no end, so it never stops.
Driving to a pose (G-3123)	Bringing a robot to a given position and heading together, such as stopping at a charging dock facing it; it needs a controller that steers on the heading still to turn as well as the distance, because aiming at the point alone leaves the arrival heading to chance.
Polar-coordinate controller (G-3124)	A pose controller that steers on the distance ρ to the goal point, the angle α from the heading to the line of sight and the angle β from the line of sight to the goal's heading: $v = k_\rho \rho$, $\omega = k_\alpha \alpha + k_\beta \beta$; near the goal the angles behave like a spring and damper, and it settles when $k_\rho > 0$, $k_\beta < 0$ and $k_\alpha > k_\rho$.
Graceful controller (Park–Kuipers smooth control law) (G-3125)	A pose controller that chooses the curvature of the arc to drive from the distance to a target pose and two angles to it, aiming the heading off the line of sight by a capped amount, and sets the speed lower on tight arcs and near the target; Nav2's `nav2_graceful_controller` implements it and Nav2's docking server uses it for the final approach.
Differential drive (G-2288)	A robot with two independently driven wheels on one axle (plus a caster); it steers by running the wheels at different speeds and can spin on the spot.

Term	Meaning
Pose (G-2280)	Where a robot is and which way it faces: on a floor, its position (x, y) and heading θ, such as (2 m, 1 m, 30°); needed because a robot at one spot drives off differently depending on its heading.
Linear velocity v (G-2291)	How fast a robot's reference point (for a differential drive, the middle of the axle) moves along its heading, in m/s; for a differential drive, the average of the two wheel speeds, so it sets how fast the robot advances.
Angular velocity ω (G-2292)	How fast a robot's heading changes, in rad/s, positive to the left; for a differential drive, the difference of the wheel speeds divided by the wheel separation.
Kinematic model (G-2297)	Equations that give how a robot's configuration changes from its speed inputs alone, ignoring masses and forces, such as $\dot{x} = v\cos\theta$, $\dot{y} = v\sin\theta$, $\dot{\theta} = \omega$; used to predict motion at low speed.
Cascade control (G-3086)	Nesting feedback loops so that the outer loop's output is the inner loop's reference, such as a heading loop that commands the yaw rate for a gyro loop; it lets each loop be designed in turn, catches disturbances early and offers a place for limits.
atan2 (G-2325)	A function atan2(rise, run) that returns the direction of a line over the full circle from its rise and run given separately; needed because the plain inverse tangent of the ratio cannot tell opposite directions apart.
Heading θ (G-2284)	The angle from the world x-axis to the direction a robot faces, measured counter-clockwise; it decides which way the robot moves when it drives forward.
Angle wrapping (G-2404)	Adding or subtracting full turns until an angle or angle difference lies between $-180°$ and 180°, so that 175° and $-178°$ count as 7° apart, not 353°; needed whenever bearings or headings are compared.
Polar coordinates (G-1511)	Describing a point by its distance r from the origin and its angle θ: $(r\cos\theta, r\sin\theta)$.

Term	Meaning
Proportional control (P control) (G-2958)	Feedback that sets the command in proportion to the control error, such as a torque of 0.2 N m per rad of heading error; it pulls the output towards the reference like a spring, but alone it swings when nothing damps the motion and leaves an error under a steady load.
Inverse kinematics (wheeled robot) (G-2295)	Working out the wheel speeds that give a wanted linear and angular velocity, used by a controller to command the motors.
Turning radius R (G-2293)	The distance from the point a robot is turning about (the instantaneous centre of curvature, ICC) to the robot's reference point; $R = v/\omega$, so it says how tight a turn is.
Dot notation $\dot{x}$ (G-2296)	A dot over a quantity means its rate of change per second (its derivative with respect to time), such as $\dot{x} = 0.433$ m/s; models of motion are written with it.
First-order system (G-2987)	A system whose output follows a differential equation with only a first derivative, such as the robot's speed under P control; it closes the same fraction of its remaining gap in every time constant and can never overshoot.
Time constant (G-2988)	The time a first-order system takes to close 63 % of the gap to its final value, m/(K_v + b) = 1 s for the robot's speed loop; it fixes the rise time (about 2.2 time constants) and the 2 % settling time (about 4).
Zero-order hold (G-3109)	Keeping each command constant from its tick until the next one, so the actuator gets a staircase; simple and cheap, but the held value is on average half a sample period old, so it acts like a delay of $T/2$.
Error dynamics (G-2986)	The differential equation the error of a controlled system obeys, found by putting the controller's rule into the plant's equation of motion, such as 0.1 e" + 0.08 e' + 0.4 e = 0 for the heading under PD control; its solution predicts the step response from the gains.

Term	Meaning
Second-order system (G-2990)	A system whose output follows a differential equation with a second derivative, such as the robot's heading under PD control; written e" + 2$\zeta\omega$_n e' + ω_n^2 e = 0, its reply is set by two numbers, the natural frequency and the damping ratio.
Natural frequency (G-2991)	The rate, in rad/s, at which a second-order system would swing with no damping, $\omega_n = \sqrt{K_p/I}$, 2 rad/s for the heading loop; it sets how fast the reply is, and raising it squeezes the whole reply in time.
Damping ratio (G-2992)	A second-order system's damping compared with the damping that just stops it from swinging, $\zeta = K_d/(2\sqrt{K_pI})$, 0.2 for D gain 0.08 and 1 for 0.4; it alone sets the shape of the reply and its overshoot.
Critically damped (G-2995)	A second-order system with damping ratio exactly 1: its characteristic equation has one double real root, so it gives the fastest step response that does not overshoot, such as the turn with K_p = K_d = 0.4 settling in 2.92 s.